SkillNavora

MODULE 4 HANDBOOK

Module 4: Generative AI and Large Language Models

8 sections, every topic explained with how it works, examples and resources.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

Module 4: Generative AI and Large Language Models

Every section below explains the concept, how it works, each listed topic in detail, a worked example and where to learn more.

Sections

1. 4.1 Generative AI Fundamentals
2. 4.2 Major AI Model Providers
3. 4.3 Prompt Engineering
4. 4.4 Context Engineering
5. 4.5 Embeddings and Tokenisation
6. 4.6 Fine-Tuning Fundamentals
7. 4.7 LLM Evaluation
8. 4.8 AI Application Development with Lovable

MODULE 4 · SECTION 4.1

4.1 Generative AI Fundamentals

DEFINITION

> Generative AI creates new text, images, audio or video. Large language models (LLMs) are transformer models trained on huge text sets to predict the next token.

HOW IT WORKS

- Text is split into tokens (word pieces). The model reads a context window of tokens and predicts the next one, repeatedly.
- Attention lets each token weigh the others in the window. Parameters are the learned weights; temperature controls how random the choice of next token is.
- Hallucination = a fluent but false answer. Multimodal models accept or produce several media types.

EVERY TOPIC EXPLAINED

Generative models; foundation models; LLMs; transformers and attention. Generative models learn the patterns of data and produce new samples. Foundation models are large models trained on broad data and adapted to many tasks. LLMs are foundation models for language. Transformers process all tokens in parallel; attention lets each token weigh the importance of every other token, which captures meaning and context.

Tokens, tokenisation, context windows, parameters, temperature. Tokens are pieces of words; tokenisation splits text into them. The context window is the maximum tokens (input + output) per request. Parameters are learned weights. Temperature controls randomness: 0 is focused and repeatable, higher values are more varied and creative.

Hallucinations; multimodal AI (text, image, audio, video). Hallucination is a confident but false or unsupported output; reduce it with retrieval (RAG), clear instructions, citations and evaluation. Multimodal AI handles several media types: reading images and PDFs, transcribing audio, generating images or video.

WORKED EXAMPLE

> You paste a 40-page contract into a model with a 200,000-token window; it fits, so the model can read the whole document in one request.

RESOURCES

- Anthropic: prompt and model docs https://docs.claude.com/
- OpenAI API docs (official) https://platform.openai.com/docs

4.2 Major AI Model Providers

DEFINITION

> Providers offer models through APIs: OpenAI, Anthropic (Claude), Google (Gemini), plus open-weight models run through Hugging Face or Ollama.

HOW IT WORKS

- Each API takes messages (system and user roles), returns text or tool calls, and bills by input and output tokens.
- Model choice weighs quality, speed, cost, context size, data location and terms. Multi-model designs route easy tasks to cheap models and hard tasks to stronger ones.
- Open-weight models can run on your own servers for privacy, at the cost of setup and maintenance.

EVERY TOPIC EXPLAINED

OpenAI API, Anthropic Claude API, Google Gemini API. OpenAI API: GPT models via the Responses and Chat Completions APIs. Anthropic Claude API: Claude models via the Messages API, strong at long documents, coding and agentic tool use. Google Gemini API: Gemini models via AI Studio or Vertex AI, with long context and multimodal input. All three use API keys, messages, tool calling and structured outputs.

Open-source models, Hugging Face, Ollama. Open-source and open-weight models (Llama, Mistral, Qwen, Gemma) can run on your own hardware. Hugging Face hosts models, datasets and libraries (transformers). Ollama runs open models locally with one command (ollama run llama3), useful for privacy and offline prototypes.

Model selection, pricing and usage considerations, limitations, multi-model architecture. Model selection compares quality on your own test set, speed, cost per million tokens, context size, data residency and terms. Pricing is per input and output token, with discounts for caching and batch. Limitations include rate limits, outages and model updates. Multi-model architecture routes each task to the best-value model and falls back if one provider fails.

WORKED EXAMPLE

> A support bot sends simple FAQ questions to a small, cheap model and escalates complex complaints to a larger model.

RESOURCES

- Google AI for Developers (official) https://ai.google.dev/
- Hugging Face Hub https://huggingface.co/docs/hub/index

4.3 Prompt Engineering

DEFINITION

> Prompt engineering is writing instructions and examples that get a model to produce the output you need reliably.

HOW IT WORKS

- Zero-shot gives the task alone; few-shot adds examples. Role prompting sets a persona or expertise.
- Structured output: ask for JSON with a named schema and validate it in code.
- System prompts set rules; user prompts carry the request. Version prompts and test them against a fixed set of cases.
- Chain-of-thought asks the model to reason step by step; for production you judge the final answer, not a hidden reasoning trace.

EVERY TOPIC EXPLAINED

Zero-shot and few-shot prompting. Zero-shot: give only the instruction ('Classify this review as positive or negative'). Few-shot: add 2–5 worked examples so the model copies the pattern and format. Few-shot is the fastest way to improve consistency.

Chain-of-thought concepts and safe reasoning guidance. Chain-of-thought asks the model to work through steps before answering, improving maths and multi-step logic. Many current models reason internally; for production, request a clear final answer and, if needed, a brief justification you can show users, and evaluate the final answer.

Role-based prompting; structured and JSON output; prompt templates. Role-based prompting sets expertise and tone ('You are a senior insurance claims reviewer'). Structured output asks for JSON matching a schema; many APIs can enforce the schema. Prompt templates insert variables ({customer_name}, {ticket_text}) into a fixed prompt so every call is consistent.

System and user instructions; prompt evaluation, optimisation and versioning. The system instruction sets lasting rules (role, tone, limits, format); the user message carries the specific request. Prompt evaluation runs the prompt on a test set and scores it. Optimisation changes one thing at a time and re-scores. Versioning stores each prompt with an ID so you know which version produced which output.

WORKED EXAMPLE

> A prompt asks for {"sentiment": "positive|negative|neutral", "reason": "..."} and includes three labelled examples; code rejects any reply that is not valid JSON.

RESOURCES

- Anthropic: prompt engineering overview https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- OpenAI prompt guidance https://platform.openai.com/docs/guides/prompt-engineering

4.4 Context Engineering

DEFINITION

> Context engineering decides what goes into the model's context window for each call: instructions, retrieved facts, memory, history and tool results.

HOW IT WORKS

- Relevant context improves answers; irrelevant or old context wastes tokens and can confuse the model.
- Techniques: retrieve only the needed chunks, summarise long history, keep a short memory of user facts, and trim tool output.
- For agents, the context is rebuilt every step from the task state, not kept as an endless transcript.

EVERY TOPIC EXPLAINED

Context windows and context management; context retrieval. The context window is everything the model can see in one call. Context management decides what goes in: instructions, the user's request, retrieved documents, tool results and history. Context retrieval fetches only relevant information from databases or documents at the moment it is needed.

Memory, conversation history, context compression, relevance. Memory keeps useful facts across turns or sessions (user preferences, earlier decisions), stored outside the model and re-inserted when relevant. Conversation history can be truncated or summarised. Context compression shortens long text while keeping key facts. Relevance ranking keeps the most useful items when space is limited.

Long-context strategies; context engineering for agents. Long-context strategies: put the most important instructions at the start and the question at the end, use clear section headings or XML-style tags, and quote the relevant passages. For agents, context engineering means giving each step only the tools, state and information it needs, and summarising results instead of passing raw logs.

WORKED EXAMPLE

> A support agent sends the current ticket, the customer's plan, and the three most relevant help articles, not the whole history.

RESOURCES

- Anthropic: effective context engineering https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
- Anthropic: building effective agents https://www.anthropic.com/engineering/building-effective-agents

4.5 Embeddings and Tokenisation

DEFINITION

> An embedding is a list of numbers that represents the meaning of text so that similar meanings get similar vectors. Tokenisation splits text into the units the model counts.

HOW IT WORKS

- Similarity is measured with cosine similarity: vectors pointing the same way score near 1.
- Token counts decide cost and context limits. A common rule of thumb is about 0.75 English words per token, but count with the provider's tokenizer for accuracy.
- Pick an embedding model by quality, dimension size, language support and price; the same model must be used for indexing and querying.

EVERY TOPIC EXPLAINED

What embeddings are and how they work; semantic representation. An embedding model turns text (or images) into a fixed-length list of numbers, for example 1,536 values. Texts with similar meaning land close together in that space. This semantic representation captures meaning rather than exact words.

Similarity search; cosine similarity. Similarity search finds the stored vectors closest to a query vector. Cosine similarity measures the angle between vectors (1 = same direction, 0 = unrelated). Other measures are dot product and Euclidean distance; use the one the embedding model recommends.

Token counting, token costs, embedding model selection. Count tokens with the provider's tokenizer (tiktoken for OpenAI, the count-tokens endpoint for Claude). Cost = input tokens × input price + output tokens × output price. Choose an embedding model by retrieval quality on your data, languages, dimension size (storage cost) and price; never mix embedding models in one index.

WORKED EXAMPLE

> The question 'how do I cancel?' and the help article 'Ending your subscription' have a high cosine similarity even though they share few words.

RESOURCES

- OpenAI: embeddings guide https://platform.openai.com/docs/guides/embeddings
- Hugging Face: sentence embeddings https://huggingface.co/docs/sentence-transformers/index

4.6 Fine-Tuning Fundamentals

DEFINITION

> Fine-tuning further trains a pretrained model on your examples so it learns a style, format or narrow skill.

HOW IT WORKS

- Prepare pairs of input and ideal output, split into train and evaluation sets, run supervised training, then compare with the base model on held-out cases.
- Use fine-tuning for format and behaviour. Use RAG for facts that change, since fine-tuning does not keep knowledge fresh and is harder to update.
- Costs include data preparation, training runs, evaluation and ongoing re-tuning.

EVERY TOPIC EXPLAINED

What fine-tuning is; fine-tuning vs RAG. Fine-tuning continues training a pretrained model on your examples so it learns a format, tone or specialised task. Fine-tuning changes behaviour; RAG supplies knowledge. Use RAG for changing facts and citations; use fine-tuning for consistent style, structured outputs or cheaper smaller models.

Supervised fine-tuning; dataset preparation; evaluation; cost and limits. Supervised fine-tuning uses input-output pairs (often hundreds to thousands, in JSONL format). Prepare clean, varied, correct examples; hold out a test set; compare the tuned model with the base model plus a good prompt. Limits: cost, need for good data, risk of forgetting general skills, and retraining when the base model changes.

WORKED EXAMPLE

> A clinic wants every summary in a fixed template; 500 example notes and summaries are used to fine-tune a model, and the output is checked against a template validator.

RESOURCES

- OpenAI fine-tuning guide https://platform.openai.com/docs/guides/fine-tuning
- Hugging Face LLM course https://huggingface.co/learn/llm-course

4.7 LLM Evaluation

DEFINITION

> Evaluation measures whether an LLM answer is correct, grounded, safe and useful, using a fixed test set and clear scoring rules.

HOW IT WORKS

- Build a test set of real questions with expected answers or rubrics. Score accuracy, relevance, faithfulness (supported by the source), format and safety.
- Automated scoring can use another model as judge; check that judge against human labels, because judges have biases.
- Track latency and cost per request alongside quality, and re-run the set after each prompt or model change.

EVERY TOPIC EXPLAINED

Accuracy, relevance, faithfulness, groundedness, hallucination evaluation. Accuracy: is the answer correct? Relevance: does it address the question? Faithfulness/groundedness: is every claim supported by the provided sources? Hallucination evaluation counts unsupported claims, often by checking each claim against the retrieved context.

Response quality, latency, cost per request. Response quality covers clarity, completeness, tone and format. Latency is time to first token and total time. Cost per request = tokens used × price. Track all three together: a better answer that is 5× slower or costlier may not be acceptable.

Human vs automated evaluation; LLM-as-a-judge limitations. Human evaluation is the gold standard but slow and costly. Automated evaluation uses exact-match rules, code checks or an LLM-as-a-judge with a rubric. Judges can favour longer answers, their own style or the first option shown, so calibrate them against human labels and use clear rubrics.

WORKED EXAMPLE

> Before release, 200 test questions are run; the release is blocked if faithfulness drops below 90% or any safety case fails.

RESOURCES

- OpenAI evals guide https://platform.openai.com/docs/guides/evals
- Anthropic: demystifying evals for AI agents https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

4.8 AI Application Development with Lovable

DEFINITION

> Lovable is an AI app builder that creates a working web app from a prompt and connects to Supabase for database, login and APIs.

HOW IT WORKS

- You describe screens and data in a prompt; Lovable writes React code and a preview you can edit in chat.
- Connect Supabase for tables and authentication; define row-level security so users see only their rows.
- Test each feature, sync to GitHub, and deploy. AI-generated code can contain security gaps: review auth, secrets and inputs before launch.

EVERY TOPIC EXPLAINED

What Lovable is; prompting Lovable; building frontends. Lovable is an AI app builder: you describe an app in plain language and it generates a React + TypeScript web app with a live preview. Prompt it like a spec: the user, the screens, the data fields and the actions. Build the frontend one feature at a time and check each one in the preview.

Connecting Supabase; database, authentication, API integration. Connect Supabase from Lovable to get a PostgreSQL database, authentication (email, Google login) and APIs. Define tables and row-level security policies. Call external APIs (including AI models) from Supabase Edge Functions so keys stay on the server.

Testing, debugging, GitHub synchronisation, deployment, limitations of AI-generated development. Test every flow (sign-up, create, edit, delete, error cases). Debug with the browser console and Lovable's chat. Sync to GitHub for version control and code review, then deploy with one click or to your own hosting. Limits: generated code may have security gaps, duplicated logic or poor structure; review it before customers use it.

WORKED EXAMPLE

> A lead-capture app: a form writes to a Supabase table, an edge function calls the LLM to score each lead, and the dashboard lists scored leads.

RESOURCES

- Lovable docs https://docs.lovable.dev/
- Supabase: row level security https://supabase.com/docs/guides/database/postgres/row-level-security